AI in Medicine I

Notes by Marco Galletti
Lecturers: Julia Schnabel, Daniel Rueckert

Technical University of Munich
Winter Semester 2026/27

Contents

1 Introduction to AI for Health **2**
1.1 From expert systems to foundation models . . . 2
1.2 Medical imaging . . . 2
1.3 Signals and clinical records . . . 3

2 Basic Deep Learning Architectures **4**
2.1 Convolutional neural networks . . . 4
2.2 Classic CNN architectures . . . 7
2.3 Evaluating classifiers and case studies . . . 9

3 Advanced Deep Learning Architectures **12**
3.1 Very deep CNNs . . . 12
3.2 Recurrent neural networks . . . 13
3.3 Transformers . . . 15

1 Introduction to AI for Health

1.1 From expert systems to foundation models

Expert systems (1970s). An **expert system** encodes medical knowledge by hand: a **knowledge base** of facts and rules, an **inference engine** that applies them to a new case, and a user interface through which the physician describes the case and receives advice with an explanation. **MYCIN** identified the bacteria causing severe infections and recommended antibiotics with a dosage adjusted to body weight, performing better than infectious-disease experts.

Probabilistic models (1980s). **INTERNIST-1/QMR** supports diagnosis in internal medicine with a probabilistic model: a bipartite directed graph of binary disease and symptom variables, with an edge from each disease to the symptoms it causes (fig. 1); diagnosis means inferring the diseases from the observed symptoms. The graph was elicited from doctors with years of manual work, and it drove advances in Bayesian networks and approximate inference. Its limits are those of every hand-built system: symptoms must be entered manually, and the model is hard to maintain and to generalise.

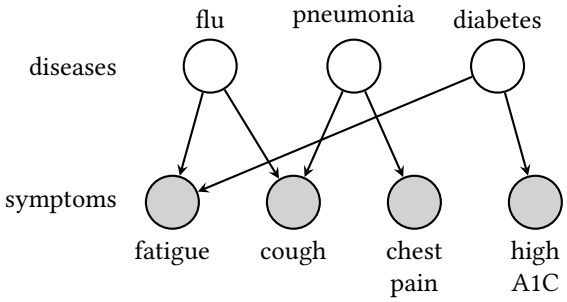


Figure 1: A small piece of a disease-symptom network in the style of INTERNIST-1/QMR.

Foundation models (2020s). A **foundation model** for **generalist medical AI** (GMAI) is trained by self-supervision on many modalities at once (images, electronic health records (EHRs), signals, omics, graphs, text, audio), uses medical knowledge (literature, clinical notes, knowledge graphs), and supports flexible interactions: the task is specified dynamically, e.g. by a question, with multimodal inputs and outputs. Examples are bedside decision support (an alert that the model explains on request), radiology reports whose findings are grounded in image regions, and assistance during procedures such as endoscopy. Deployment requires approval, validation, audits and analyses of bias and fairness.

Large language models. An LLM such as ChatGPT is a GPT model pretrained on large text corpora, refined by **reinforcement learning from human feedback** (RLHF) and wrapped in **guardrails** for safety. In medicine it can serve patient care (documentation, summarisation, medical knowledge), education and research.

1.2 Medical imaging

The goal is a model that integrates all data about a patient (images, vital signs, lab tests, genomics and other omics, wearables, clinical records and history, lifestyle) and outputs a diagnosis or prognosis. Imaging visualises organs and tissue inside the body: an **imaging modality** produces a map of some physical property of the tissue.

Definition 1.1 (Image)**.** An **image** is a function of the position: $f(x, y)$ in 2D, $f(x, y, z)$ in 3D, or $f(x, y, z, t)$ in 4D (3D over time). Its values are scalars, displayed as intensity, or vectors, which are harder to visualise.

Modalities differ in the physical mechanism that generates contrast (radiation, magnetisation, sound, light) and in their risks: some use harmful **ionising radiation**, some are **invasive** (a contrast agent is injected or a device inserted into the body), many are harmless and non-invasive.

- **X-ray radiography**: ionising radiation; a 2D projection of the 3D body.
- **Computed tomography** (CT): X-rays restricted by a collimator to one or a few slices; source and detector rotate around the patient, and the slice image $I(x, y) = \mu(x, y, z_0)$ (μ the X-ray attenuation) is computed from the projections taken at many angles. Stacked slices give a 3D volume.
- **Magnetic resonance imaging** (MRI): by varying the acquisition it maps different tissue properties of the same anatomy (**T1-weighted**, **T2-weighted**, **proton density** images).
- **Ultrasound** (US): images flow via the Doppler effect; its main noise is **speckle**, which depends on the signal.
- **SPECT** and **PET**: nuclear imaging of an injected radioactive tracer; functional, low resolution.

Anatomical imaging shows structure, **functional imaging** shows physiological activity; hybrid modalities such as **PET/CT** overlay function (PET) on anatomy (CT), as in fig. 2. Table 1 summarises the properties.

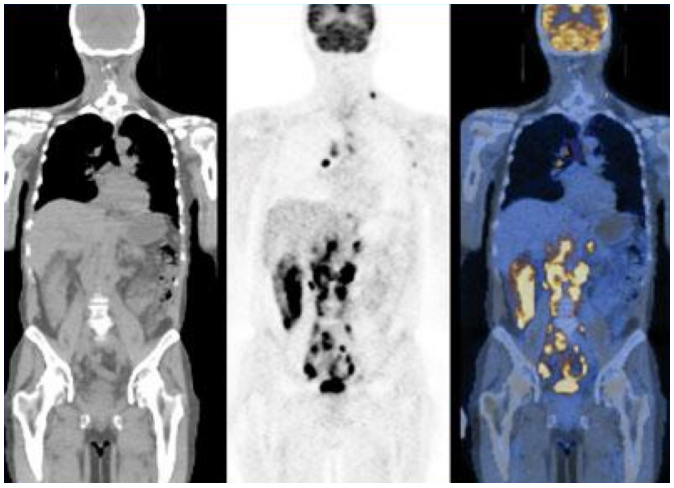


Figure 2: Whole-body PET/CT: CT (anatomy), PET (function, dark spots of high tracer uptake), and their overlay.

	2D	3D	4D	Ionising	Invasive	Anatomical	Functional	High res.
X-ray	✓	×	×	✓	×	✓	×	✓
Ultrasound	✓	✓	✓	×	×	✓	✓	✓
CT	n/a	✓	✓	✓	×	✓	×	✓
MRI	✓	✓	✓	×	×	✓	✓	✓
SPECT	n/a	✓	×	✓	✓	×	✓	×
PET	n/a	✓	✓	✓	✓	×	✓	×

Table 1: Comparison of imaging modalities.

1.3 Signals and clinical records

Signals. Wearables, **electroencephalography** (EEG, electrical activity of the brain) and **electrocardiography** (ECG, of the heart) produce multichannel time series, represented in the time domain, the frequency domain (spectral representation) or a graph domain (state representation). For example, an **inertial measurement unit** (IMU) worn on the body records velocity, acceleration and displacement, and a classifier maps them to an activity (walk, fall, stand, sit); for EEG and ECG a typical task is to

detect abnormal recordings.

Records and standards. Large public datasets combine these sources: **MIMIC** holds de-identified intensive-care records (vital-sign time series, lab values, medication, images, free-text reports), and **UK Biobank** is a population cohort with imaging, genetics, lifestyle and clinical data. Standard codes make records comparable and machine-readable:

- **ICD-10** (International Classification of Diseases) codes diseases hierarchically: a letter and two digits give the category (the letter range gives the chapter, e.g. neoplasms or injuries), and further characters encode etiology, location, laterality and extension;
- **LOINC** identifies laboratory observations, **NDC** (National Drug Codes) identifies drugs;
- **UMLS** (Unified Medical Language System) maps between medical vocabularies, a thesaurus and ontology of biomedical concepts that enables natural language processing.

2 Basic Deep Learning Architectures

Typical imaging tasks are **image classification** (a categorical label: modality such as MR, CT or angiography, MR sequence such as T1, T2 or PD, image quality pass/fail, disease state such as Alzheimer's disease versus normal), **image regression** (a quantity: the patient's age, a quality score in $[0, 1]$) and **region classification** (classifying sub-regions of an image: anomaly detection, anatomy or organ detection). The classical pipeline first computes hand-crafted features $\boldsymbol{x} = f(\mathbf{I})$ of the image $\mathbf{I}$ (raw pixel values, SIFT, HoG, textons) and then trains a model $h_\Theta(\boldsymbol{x})$ on them; a deep network learns the whole map $h_\Theta(\mathbf{I})$ from the image to the prediction, features included.

2.1 Convolutional neural networks

Neurons. A neuron computes a weighted sum of its inputs and applies a non-linear **activation function** f_a (fig. 3):

$$\hat{y}(\boldsymbol{x}) = f_a\Big(\sum_{i=1}^{N} x_i\theta_i + \theta_0\Big),$$

where the bias θ_0 is the weight of a constant input $x_0 = 1$. Common activations are the sigmoid $S(x) = 1/(1 + e^{-x})$ with values in $(0, 1)$, $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ with values in $(-1, 1)$, and the rectifier (**ReLU**) $f(x) = \max(0, x)$. Training minimises a loss over the data $(\mathbf{X}, \mathbf{Y})$, e.g. the mean squared error, by gradient descent with step size $\eta > 0$:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \qquad \theta^* = \arg\min_\theta \mathcal{L}(\theta, \mathbf{X}, \mathbf{Y}), \qquad \theta_i^{t+1} = \theta_i^t - \eta\,\frac{\partial \mathcal{L}}{\partial \theta_i^t}.$$

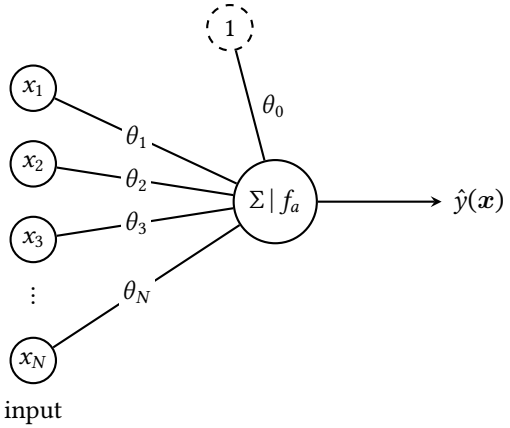


Figure 3: A neuron: weighted sum of the inputs plus bias, followed by the activation f_a.

From fully connected to convolutional layers. In a fully connected network each neuron of the first layer sees every pixel and detects one pattern, so it needs one weight per pixel: $512^2 = 262{,}144$ weights for a 512×512 image. Two properties of images reduce this (fig. 4).

- **Locality**: features are local, only neighbouring pixels are correlated. In a **locally connected network** each neuron sees only a small window, its **kernel** (a 10×10 kernel has 100 weights), but it detects its pattern only at its own location.
- **Stationarity**: the same feature may appear anywhere in the image. So all neurons share the same kernel weights (**weight sharing**), and sliding one kernel over the image is a convolution.

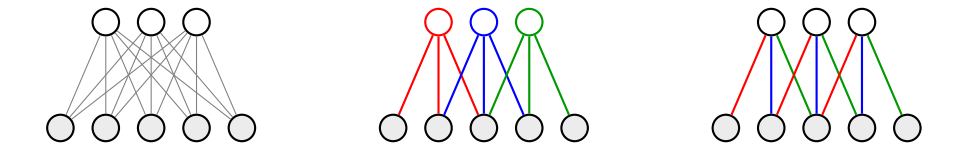


Figure 4: From fully connected to convolution (1D, kernel of size 3); colours mark distinct weights.

The single neuron $a(\boldsymbol{x}) = f_a(\sum_i \theta_i x_i + b)$ thus becomes a **feature map** $\mathbf{a}(\boldsymbol{x}) = f_a(\boldsymbol{\theta} * \boldsymbol{x} + b)$, which is itself an image.

Definition 2.1 (2D convolution)**.** For an image f and a kernel g of size $n \times n$ (indices $0, \dots, n-1$),

$$(f * g)[x, y] = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} f\left(x - i + \left\lfloor \frac{n}{2} \right\rfloor,\ y - j + \left\lfloor \frac{n}{2} \right\rfloor\right) g(i, j);$$

the shift by $\lfloor n/2 \rfloor$ centres the kernel on the pixel (x, y).

Fixed kernels extract simple features: horizontal edges, vertical edges (the transpose) and, with the Laplacian, edges in every direction,

$$\begin{pmatrix} -1 & -1 & -1 \\ 1 & 1 & 1 \\ -1 & -1 & -1 \end{pmatrix}, \qquad \begin{pmatrix} -1 & 1 & -1 \\ -1 & 1 & -1 \\ -1 & 1 & -1 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

A CNN learns its kernels instead.

Convolutional layers. A **convolutional layer** learns D kernels at once and stacks the D resulting feature maps into a D-channel image (like the RGB channels of a colour image). If its input has C channels, each kernel extends over the full depth: it has size $k_1 \times k_2 \times C$ and sums over the channels (fig. 5). Deeper layers process the feature maps of the previous layer and combine them into more complex representations. A layer has

$$k_1 k_2 C D \text{ weights (plus } D \text{ biases)},$$

e.g. $5 \cdot 5 \cdot 3 \cdot 4 = 300$ weights for $D = 4$ filters of size 5×5 on an RGB input ($C = 3$). The count does not depend on the image size, and each weight is used, hence trained, at every position, so it sees much more training data.

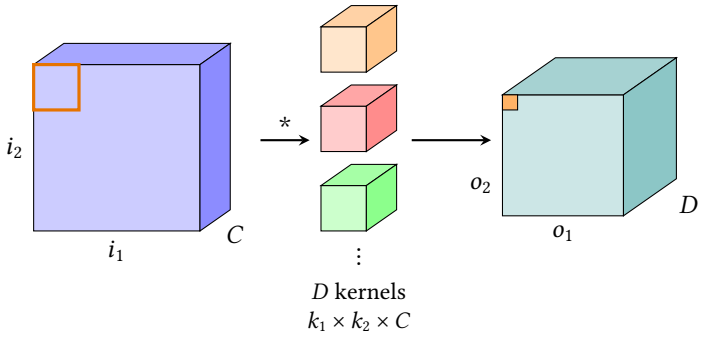


Figure 5: A convolutional layer: each of the D kernels spans all C input channels and produces one output feature map.

Definition 2.2 (Output size of a convolution)**.** Along axis j let i_j be the input size, k_j the kernel size, p_j the **zero padding** (zeros added at both the beginning and the end of the axis) and s_j the **stride** (distance between consecutive kernel positions). The feature map has size

$$o_j = \left\lfloor \frac{i_j - k_j + 2p_j}{s_j} \right\rfloor + 1.$$

Indeed the padded axis has length $i_j + 2p_j$, and the kernel fits at the positions $0, s_j, 2s_j, \dots, ms_j$ with $ms_j + k_j \le i_j + 2p_j$, so $m = \lfloor (i_j + 2p_j - k_j)/s_j \rfloor$ and $o_j = m + 1$. Without padding and stride ($p_j = 0$, $s_j = 1$) this is $o_j = i_j - k_j + 1$ (e.g. $5 - 3 + 1 = 3$); with padding only, $o_j = i_j - k_j + 2p_j + 1$ (e.g. $i_j = 5$, $k_j = 4$, $p_j = 2$ give 6); fig. 6 shows a case with stride.

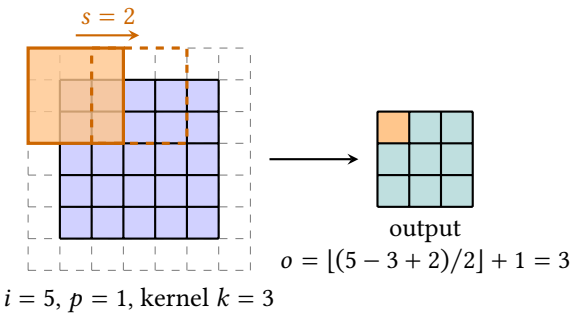


Figure 6: Convolution with padding and stride: the kernel jumps by $s = 2$ over the zero-padded input.

Receptive field. The **receptive field** of an output pixel is the input region it depends on; it grows with depth. A $k \times k$ convolution without padding and with stride 1 shrinks the map by $k - 1$ and enlarges the receptive field by $k - 1$, so after L such layers the receptive field is $L(k - 1) + 1$ (fig. 7). For instance four 5×5 layers map a 17×17 input to 13, 9, 5 and finally 1 pixel, whose receptive field is the whole 17×17 input. Small filters in many layers use fewer parameters than one large filter: $4 \cdot 25 = 100$ against $17^2 = 289$ weights (per pair of input and output channels). Without padding the feature maps shrink with depth.

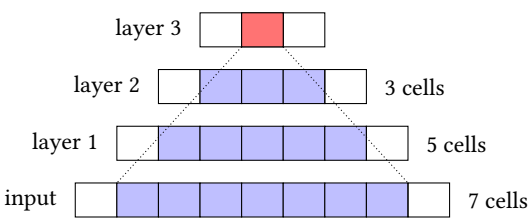


Figure 7: Receptive field in 1D: three convolutions with $k = 3$; the red pixel sees 3, 5, 7 cells below it.

Pooling. Downsampling reduces the size (memory) of deeper layers, gives invariance to small translations, and the contraction forces the network to learn high-level features. A **pooling** operator has a kernel size k and a stride s (the output size follows the same formula as a convolution) and is applied to each feature map separately, so the number of channels is unchanged: e.g. $224 \times 224 \times 10$ becomes $112 \times 112 \times 10$ with $k = s = 2$. Its kernel function, e.g. **max pooling** or **mean pooling**, has no parameters (fig. 8). Replacing the fixed function by learnable weights $w_1, \dots, w_{k^2}$ turns pooling into a strided convolution, i.e. a learned downsampling.

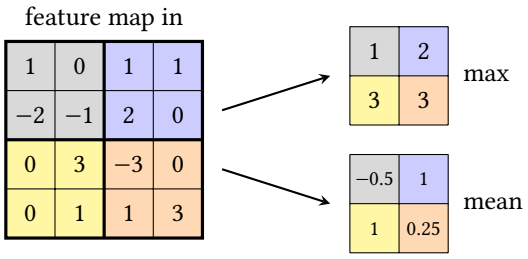


Figure 8: Max and mean pooling with $k = s = 2$: each coloured 2×2 block gives one output value.

Upsampling. To enlarge a feature map by a factor s, insert $s - 1$ zeros between neighbouring entries, pad, and convolve. With a fixed kernel this is interpolation: the kernel with rows $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$, $(\frac{1}{2}, 1, \frac{1}{2})$, $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$ copies the original values and fills each new entry with the mean of its two or four original neighbours, i.e. bilinear interpolation (fig. 9). A **transposed convolution** (or **deconvolution**) learns this kernel, a data-driven upsampling. Its kernel size k sets how much context influences each output value, its stride s is the upsampling factor, and its padding p must be chosen carefully to obtain exactly the wanted output shape. Equivalently, a deconvolution with (k, s, p) goes back from the output shape of the strided convolution with the same (k, s, p) to that convolution's input shape.

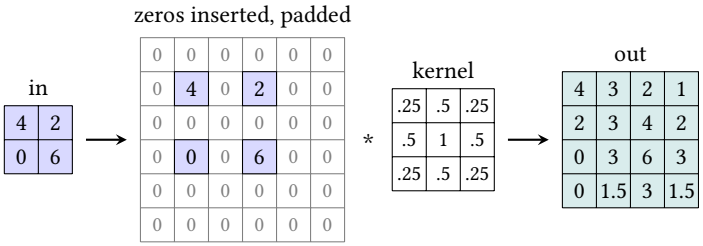


Figure 9: Upsampling a 2×2 map with $s = 2$: zero insertion, then a 3×3 convolution (here the fixed bilinear kernel).

2.2 Classic CNN architectures

LeNet. The first successful CNN on a real-world problem (handwritten digit recognition) set the pattern later architectures follow: a **feature-learning** part of alternating convolution and subsampling layers, then a **classification** part of fully connected layers (an MLP), as in fig. 10. It subsamples by average pooling, uses tanh activations, and connects two of its convolutional stages sparsely (each feature map sees only a subset of the previous maps).

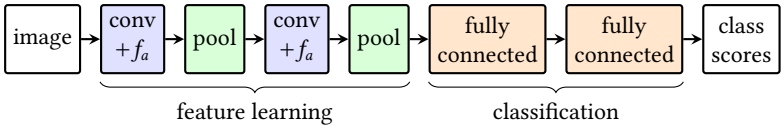


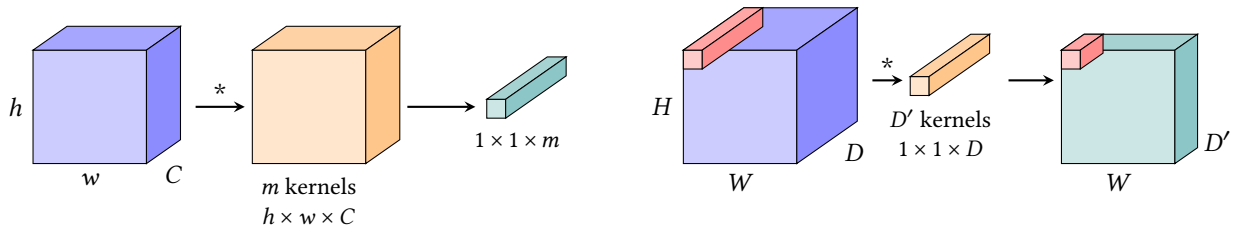
Figure 10: Basic CNN pattern: convolution and pooling learn features, fully connected layers classify.

AlexNet. A much larger LeNet whose novelties are ReLU activations and max pooling, convolutional layers stacked directly on top of each other (instead of a pooling layer after every convolution), **dropout** regularisation, **data augmentation** (random transformations and intensity variations) and training on GPUs. Its first-layer kernels turn out to be oriented edge detectors and colour blobs. More generally, features get more abstract with depth: edges and blobs in the first layer, textures in the middle, object parts in deeper layers, object classes at the output.

VGGNet. Studies the effect of depth with a uniform design: only 3×3 convolutions with stride 1 and 2×2 max pooling, followed by three fully connected layers. It is very large, about 138 million parameters, and roughly 103 million of them sit in the first fully connected layer alone ($7 \cdot 7 \cdot 512$ inputs times 4096 units): the size of CNNs comes mostly from their fully connected layers.

Fully convolutional networks. A fully connected neuron on C feature maps of size $h \times w$ computes a weighted sum of all hwC activations plus a bias, which is exactly a convolution with one $h \times w \times C$ kernel without padding, producing a 1×1 output (fig. 11a); m such neurons are m kernels, i.e. m feature maps of size 1×1. Replacing every fully connected layer in this way gives a **fully convolutional network**, which accepts inputs of any size: a larger input yields larger feature maps and a map of predictions instead of a single one. The receptive field of each prediction stays the same, and each prediction depends only on its own receptive field, not on the whole input. With the four 5×5 layers above, a 25×25 input gives a 9×9 output ($25 - 4 \cdot 4 = 9$).

1×1 **convolutions.** **Network in network** inserts small MLPs (fully connected layers) between convolutional layers, applied at every position; this is the same as a 1×1 convolution. A 1×1 convolution with D' filters maps an $H \times W \times D$ volume to $H \times W \times D'$: each output value $f_a\left(\sum_{c=1}^{D} w_c\, a_c(x, y) + b\right)$ combines the D channels at the same pixel (fig. 11b). With $D' < D$ it reduces the number of channels cheaply.



(a) fully connected layer = convolution

(b) 1×1 convolution, $D' < D$

Figure 11: (a) A fully connected layer is a convolution with kernels as large as the input. (b) A 1×1 convolution maps the D values of a pixel to D' values.

GoogLeNet (Inception v1). A deeper network with **global average pooling** (each final feature map averaged to one value) as its final layer, and **auxiliary classifiers** attached to intermediate layers, used only during training. It is built from **Inception modules** (fig. 12), derived from the network-in-network idea: 1×1, 3×3 and 5×5 convolutions and 3×3 max pooling run in parallel on the same input and their outputs are concatenated along the channels (split-transform-merge), so the network *learns* which filter size to use. The 1×1 convolutions before the large kernels and after the pooling are **bottleneck** layers that reduce the number of channels; the module thus has the representational power of large dense layers with far fewer parameters and operations.

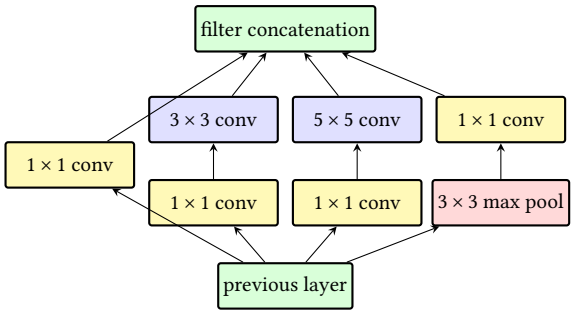


Figure 12: Inception module: parallel branches with 1×1 bottlenecks, concatenated along the channels.

Example 2.3 (Cost of a bottleneck)**.** Map a $14 \times 14 \times 480$ volume to $14 \times 14 \times 48$ with 5×5 convolutions (size-preserving padding). Each output value needs $5 \cdot 5 \cdot 480$ multiplications, so directly

$$(14 \cdot 14 \cdot 48)\,(5 \cdot 5 \cdot 480) \approx 112.9\text{M operations.}$$

With a 1×1 convolution to 16 channels first, the cost is $(14 \cdot 14 \cdot 16)(1 \cdot 1 \cdot 480) \approx 1.5$M for the 1×1 layer plus $(14 \cdot 14 \cdot 48)(5 \cdot 5 \cdot 16) \approx 3.8$M for the 5×5 layer, 5.3M in total: about 21 times fewer.

Inception v2 and v3. Inception v2 factorises large convolutions. A 5×5 (or 7×7) convolution is replaced by a stack of 3×3 ones: two 3×3 layers have a 5×5 receptive field with $2 \cdot 9 = 18$ instead of 25 weights. An $n \times n$ convolution is replaced by a $1 \times n$ followed by an $n \times 1$ one ($2n$ instead of n^2 weights). Grid-size reduction is done efficiently by parallel stride-2 convolution and stride-2 pooling branches whose outputs are concatenated.

Inception v3 adds the RMSProp optimiser, factorised 7×7 convolutions, batch normalisation in the auxiliary classifiers and **label smoothing**, a regularisation added to the loss that copes with label noise and prevents the network from becoming too confident about one class, which reduces overfitting. Even deeper networks (ResNet) belong to the advanced architectures.

2.3 Evaluating classifiers and case studies

Precision, accuracy, robustness. For a measurement, **precision** describes the random errors (statistical variability): the repeatability or reproducibility of the measurement. **Accuracy** describes the systematic errors (statistical bias), the difference between the result and the true value, which ISO calls *trueness*; ISO alternatively defines accuracy as the combination of both random and systematic errors, so that high accuracy requires both high precision and high trueness. **Robustness** is how much performance degrades as noise levels or imaging artefacts vary. (Precision as a classification metric, below, is a different quantity.)

Confusion matrix. For a binary classifier, compare the actual condition with the predicted one:

	predicted positive (PP)	predicted negative (PN)
actual positive (P)	true positive (TP), hit	false negative (FN), miss, type II error
actual negative (N)	false positive (FP), false alarm, type I error	true negative (TN), correct rejection

so $\mathrm{P} = \mathrm{TP} + \mathrm{FN}$, $\mathrm{N} = \mathrm{TN} + \mathrm{FP}$, $\mathrm{PP} = \mathrm{TP} + \mathrm{FP}$, $\mathrm{PN} = \mathrm{FN} + \mathrm{TN}$.

Definition 2.4 (Basic metrics)**.**

$$\mathrm{ACC} = \frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{P}+\mathrm{N}}, \qquad \mathrm{PPV} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \qquad \mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{P}} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \qquad \mathrm{TNR} = \frac{\mathrm{TN}}{\mathrm{N}} = \frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}}.$$

Accuracy (ACC) is the fraction of correct decisions; **precision** or **positive predictive value** (PPV) answers "how many selected items are relevant?"; **recall**, **sensitivity**, **hit rate** or **true positive rate**

(TPR) answers "how many relevant items are selected?"; **specificity** or **true negative rate** (TNR) is the analogue for the negatives.

Other common metrics are the **false positive rate** $\mathrm{FPR} = \mathrm{FP}/\mathrm{N} = 1 - \mathrm{TNR}$, the **false negative rate** $\mathrm{FNR} = \mathrm{FN}/\mathrm{P} = 1 - \mathrm{TPR}$, the **negative predictive value** $\mathrm{NPV} = \mathrm{TN}/\mathrm{PN}$, the **balanced accuracy** $(\mathrm{TPR} + \mathrm{TNR})/2$, and the F_1 **score**, the harmonic mean of precision and recall; with $\mathrm{PPV} = \mathrm{TP}/\mathrm{PP}$ and $\mathrm{TPR} = \mathrm{TP}/\mathrm{P}$,

$$F_1 = \frac{2\,\mathrm{PPV}\cdot\mathrm{TPR}}{\mathrm{PPV}+\mathrm{TPR}} = \frac{2\,\mathrm{TP}^2/(\mathrm{PP}\cdot\mathrm{P})}{\mathrm{TP}\,(\mathrm{P}+\mathrm{PP})/(\mathrm{PP}\cdot\mathrm{P})} = \frac{2\,\mathrm{TP}}{\mathrm{P}+\mathrm{PP}} = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN}}.$$

ROC curve and AUC. A classifier that outputs a score decides by comparing it with a threshold, and every threshold gives one point $(\mathrm{FPR}, \mathrm{TPR})$. The **receiver operating characteristic** (ROC) curve traces these points as the threshold varies (fig. 13). The perfect classifier sits at $(0, 1)$, random guessing gives points on the diagonal $\mathrm{TPR} = \mathrm{FPR}$, and curves closer to the top-left corner are better. The **area under the ROC curve** (AUC) lies in $[0, 1]$: it is 0 for a model whose predictions are all wrong, 1 for one whose predictions are all correct, and $\frac{1}{2}$ for random guessing (the triangle below the diagonal). It is **scale-invariant**, measuring how well the predictions are ranked rather than their absolute values, and **classification-threshold-invariant**, measuring the quality of the predictions whatever threshold is chosen.

Example 2.5 (Four classifiers in ROC space)**.** Each classifier is tested on 100 positives and 100 negatives.

	TP	FN	FP	TN	TPR	FPR	PPV	F_1	ACC
A	63	37	28	72	0.63	0.28	0.69	0.66	0.68
B	77	23	77	23	0.77	0.77	0.50	0.61	0.50
C	24	76	88	12	0.24	0.88	0.21	0.23	0.18
C′	76	24	12	88	0.76	0.12	0.86	0.81	0.82

For instance, for A: $\mathrm{PPV} = 63/91$, $F_1 = 126/191$, $\mathrm{ACC} = 135/200$. B lies on the diagonal, so it is no better than chance. C lies below it, i.e. worse than chance; C′ inverts every decision of C, which swaps TP with FN and FP with TN, so $\mathrm{TPR}' = 1 - \mathrm{TPR} = 0.76$ and $\mathrm{FPR}' = 1 - \mathrm{FPR} = 0.12$: the point is mirrored through $(\frac{1}{2}, \frac{1}{2})$ and C′ is the best of the four.

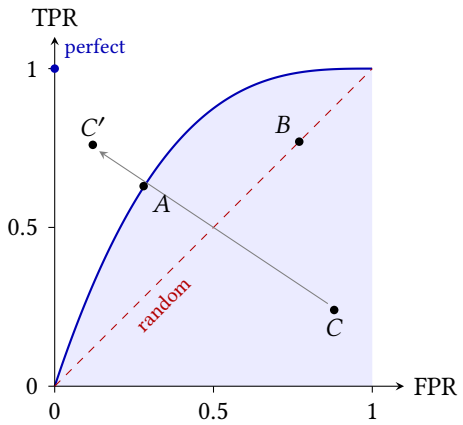


Figure 13: ROC space with the classifiers of example 2.5 and an illustrative ROC curve; the shaded area is its AUC ($= \frac{3}{4}$).

Chest radiographs. A chest X-ray (CXR) shows the lungs and the heart; radiologists compare it with a mental model of a normal CXR, looking e.g. for an enlarged heart (**cardiomegaly**) or for fluid

or air where they should not be. In **CheXpert** a CNN takes the frontal and lateral views and outputs a probability for each of 14 observations (multi-label classification). The labels (positive, negative, uncertain) are extracted automatically from the radiology reports, and the model is compared with radiologists by placing their operating points on the ROC and precision–recall curves of each pathology.

Mammography. X-ray imaging of the breast for the early detection of breast cancer. Radiologists grade lesions with **BI-RADS**, five classes of malignancy risk from 1 (lowest) to 5 (highest) based on shape, density and margin of a lesion, calcifications, distortion and asymmetry, and grade the breast with the **ACR density** classes a (almost entirely fatty) to d (extremely dense); dense breasts lower the sensitivity of mammography. AI tasks are lesion detection, malignant-versus-benign classification and multi-class grading (BI-RADS, ACR). Earlier computer-aided diagnosis (CAD) systems failed in clinical use (no real gain for radiologists, many false positives, poor workflow integration); deep learning reaches human-level performance, but generalisability, workflow integration and interpretability remain open.

Brain age regression. Healthy ageing changes brain volumes following a specific pattern. In Alzheimer's disease (AD) the connections between neurons stop working, which leads to atrophy, i.e. loss of brain volume (fig. 14). The **brain-age gap**, predicted (biological) brain age minus chronological age, is therefore a potential biomarker. A regression model, typically a CNN on whole-brain 2D or 3D MR images, is trained on healthy brains to predict the chronological age, minimising the **mean absolute error** $\mathrm{MAE} = \frac{1}{N}\sum_i |y_i - \hat{y}_i|$. At test time a brain age larger than the chronological age indicates accelerated ageing and points to a possible AD patient.

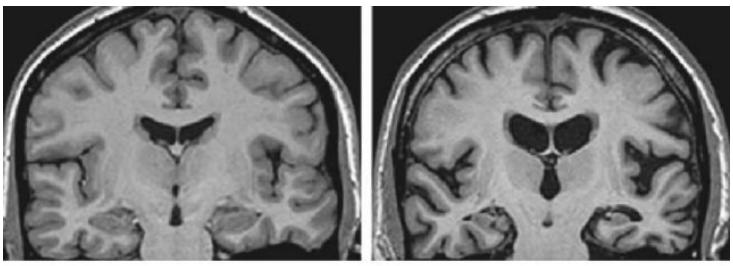


Figure 14: Coronal T1-weighted MR of a healthy and of an Alzheimer's brain: atrophy shows as enlarged ventricles and widened sulci. Source: J. M. Fernández Montenegro, *Alzheimer's Disease Diagnosis Based on Cognitive Methods in Virtual Environments and Emotions Analysis*, 2018.

Bias correction. Age predictions are biased, possibly because of regression dilution: the ages of younger subjects are overestimated and those of older subjects underestimated. The brain-age delta $\Delta = Y_{\text{predicted}} - Y_{\text{chronological}}$ is modelled as a linear function of the chronological age, $\Delta = \alpha\, Y_{\text{chronological}} + \beta$ (slope α, intercept β), and the prediction is corrected by

$$Y_{\text{corrected}} = Y_{\text{predicted}} - (\alpha\, Y_{\text{chronological}} + \beta),$$

so the chronological age enters as a covariate.

A patch-based variant trains one 3D CNN per patch of the brain and combines the P patch predictions y_i in an **ensemble**, by averaging, $y = \frac{1}{P}\sum_{i=1}^{P} y_i$, or by linear regression, $y = w_0 + \sum_{i=1}^{P} w_i y_i$; the per-patch errors also show which brain regions carry most information about age.

3 Advanced Deep Learning Architectures

3.1 Very deep CNNs

The degradation problem. Is a better network just a deeper one? Stacking more plain layers eventually makes things worse: a much deeper plain CNN has a higher *training* error, and hence also a higher test error, than a shallower one, so the problem is not overfitting. A deeper model should not do worse on the training set: copying the layers of a trained shallower model and setting the extra layers to the identity gives a deeper model with the same training error. The deeper solution exists by construction, but the solvers fail to find it, an optimisation difficulty.

ResNet. A **residual block** adds an identity shortcut around a few weight layers (fig. 15). If $H(\boldsymbol{x})$ is the mapping the block should represent, the layers learn the **residual**

$$F(\boldsymbol{x}) = H(\boldsymbol{x}) - \boldsymbol{x}, \qquad \text{so that} \qquad H(\boldsymbol{x}) = F(\boldsymbol{x}) + \boldsymbol{x}.$$

An identity block now only needs $F = 0$, which is easy to reach, so extra layers can no longer hurt the training error. With residual blocks the deeper network is again the better one (a deeper ResNet beats a shallower one, whereas among plain networks the deeper one is worse), and depth can grow to more than a hundred layers.

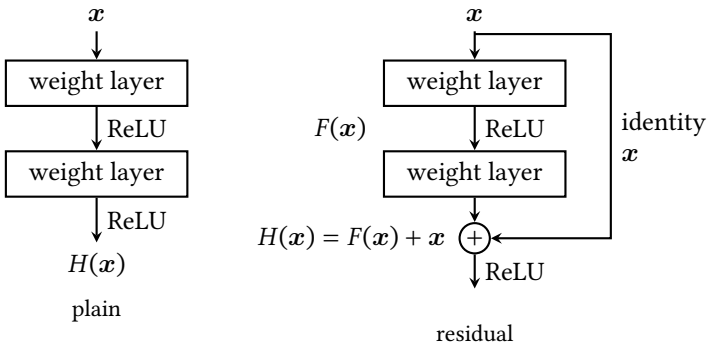


Figure 15: A plain block learns $H(\boldsymbol{x})$ directly; a residual block learns $F(\boldsymbol{x}) = H(\boldsymbol{x}) - \boldsymbol{x}$ and adds the input back.

DenseNet. Inside a **dense block** every layer receives the feature maps of *all* preceding layers, joined by channel-wise concatenation instead of the element-wise addition of ResNet (fig. 16):

$$\boldsymbol{x}_\ell = H_\ell\big([\boldsymbol{x}_0, \boldsymbol{x}_1, \dots, \boldsymbol{x}_{\ell-1}]\big).$$

Each layer adds only k new feature maps, the **growth rate**. Its advantages:

- **parameter and computational efficiency**: a ResNet layer maps C channels to C channels, $O(C \cdot C)$ parameters, whereas layer ℓ of a dense block maps about ℓk channels to k, $O(\ell \cdot k \cdot k)$ parameters, with $k \ll C$;
- **implicit deep supervision**: every layer has a short path to the loss, so the error signal reaches early layers directly (strong gradient flow);
- **more diversified features**: a layer sees all earlier features rather than a sum of correlated ones.

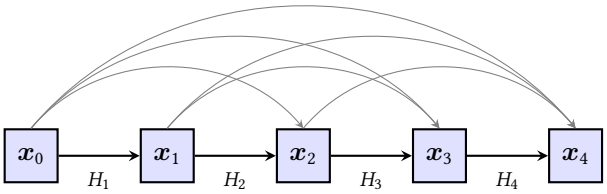


Figure 16: Dense block: each layer H_ℓ gets the concatenation of all earlier feature maps.

CheXNet. A DenseNet-121 (dense blocks separated by convolution and pooling layers, then a linear layer) classifies chest X-rays into 14 pathologies (e.g. pneumonia, cardiomegaly, effusion, pneumothorax, edema), trained on a large labelled dataset. The output is a vector of independent binary labels, one per class c, trained with the sum of binary cross-entropies

$$L(X, y) = \sum_{c=1}^{14} \Big[-y_c \log p(Y_c = 1 \mid X) - (1 - y_c) \log p(Y_c = 0 \mid X) \Big].$$

For pneumonia it was compared with radiologists through the F_1 score (the harmonic mean of precision and recall), and reached a higher F_1 than their average. More generally, modern CNN architectures trade accuracy against size and computation: some are optimised for accuracy, others (e.g. MobileNet, ShuffleNet) for a small number of parameters.

3.2 Recurrent neural networks

Sequences. Feed-forward networks map one input to one output, but much medical data is time-dependent: speech or text (e.g. healthcare records), video (e.g. ultrasound sequences), signals (e.g. ECG). Temporal context matters, and feeding a whole sequence into a feed-forward network at once is memory-inefficient, hard to train and mixes the spatial and temporal dimensions. A **recurrent neural network** (RNN) instead has a loop that lets information persist from one time step to the next, a memory used in particular in language models.

Definition 3.1 (RNN)**.** An RNN has an input, a recurrent hidden and an output layer. It reads a sequence $\dots, \boldsymbol{x}_{t-1}, \boldsymbol{x}_t, \boldsymbol{x}_{t+1}, \dots$ with $\boldsymbol{x}_t \in \mathbb{R}^N$; the hidden state $\boldsymbol{h}_t \in \mathbb{R}^M$ (the memory) and the output $\boldsymbol{y}_t \in \mathbb{R}^P$ are

$$\boldsymbol{h}_t = f_H\big(\mathbf{W}_{IH}\boldsymbol{x}_t + \mathbf{W}_{HH}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h\big), \qquad \boldsymbol{y}_t = f_O\big(\mathbf{W}_{HO}\boldsymbol{h}_t + \boldsymbol{b}_o\big),$$

with input-to-hidden weights $\mathbf{W}_{IH}$, recurrent weights $\mathbf{W}_{HH}$ (also $\mathbf{W}_{\text{rec}}$), hidden-to-output weights $\mathbf{W}_{HO}$, biases $\boldsymbol{b}_h, \boldsymbol{b}_o$, hidden activation f_H (usually tanh) and output activation f_O (usually the sigmoid σ).

Unrolling the loop gives one copy of the unit per time step, all with the same weights, starting from an initial state $\boldsymbol{h}_0$ (fig. 17); stacking such units gives a **deep RNN**.

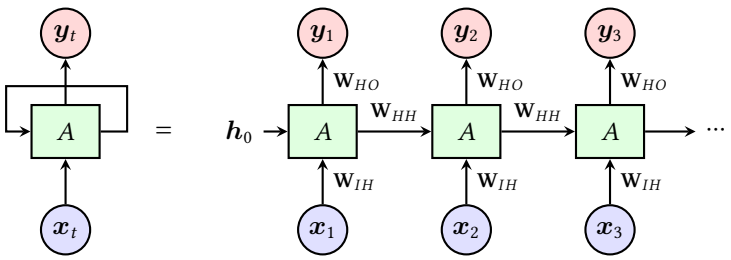


Figure 17: An RNN unit and its unrolled form: the same weights at every time step.

Backpropagation through time. For a sequence $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$ the forward pass computes $\boldsymbol{h}_t$ and $\hat{\boldsymbol{y}}_t$ for $t = 1, \dots, T$, and the loss is $L(\hat{\boldsymbol{y}}, \boldsymbol{y}) = \sum_{t=1}^{T} L(\hat{\boldsymbol{y}}_t, \boldsymbol{y}_t)$. Its gradient with respect to $\Theta = (\mathbf{W}_{IH}, \mathbf{W}_{HH}, \mathbf{W}_{HO}, \boldsymbol{b}_h, \boldsymbol{b}_o, \boldsymbol{h}_0)$ is computed backwards from $t = T$ to $t = 1$, and the weights are updated by $\Theta \leftarrow \Theta - \eta \nabla\Theta$. Both passes run sequentially through the whole sequence, so computation is slow for long sequences.

Vanishing and exploding gradients. By the chain rule the gradient contains long products in which the recurrent weights appear over and over: with $\boldsymbol{o}_t = \mathbf{W}_{IH}\boldsymbol{x}_t + \mathbf{W}_{HH}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h$,

$$\frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_1} = \prod_{t=2}^{T} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_{t-1}} = \prod_{t=2}^{T} \operatorname{diag}\big(f_H'(\boldsymbol{o}_t)\big)\, \mathbf{W}_{HH}.$$

Multiplying many factors smaller than 1 gives ever smaller numbers (**vanishing gradients**, $\mathbf{W}_{HH}$ small); factors larger than 1 give ever larger ones (**exploding gradients**, $\mathbf{W}_{HH}$ large). Remedies:

- against exploding gradients: **truncated backpropagation** (stop after a certain number of steps, so some weights are never updated), penalising or artificially reducing the gradient, **gradient clipping** (a maximum limit on the gradient);
- against vanishing gradients: a suitable weight initialisation, echo state networks, and LSTMs.

A related problem is that the last hidden state hardly depends on the first tokens of a long sequence: information from long ago is difficult to access. Besides LSTMs and Transformers, a **bidirectional RNN** helps: two hidden layers running in opposite directions feed the same output, which thus receives information from both past and future states (fig. 18), at the cost of a more complex backpropagation.

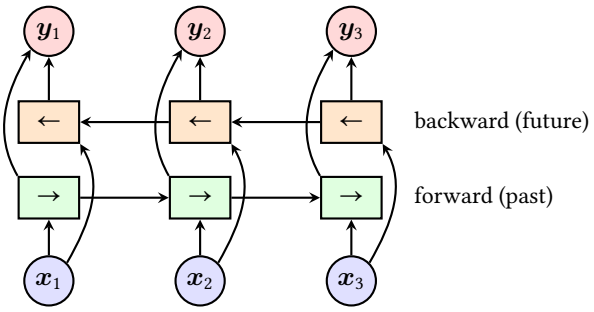


Figure 18: Bidirectional RNN: each output combines a forward and a backward hidden state.

LSTM. A **long short-term memory** unit adds a **cell state** $\boldsymbol{c}_t$, a pipeline flowing through time that is changed only by element-wise operations (fig. 19); put simply, along this path the recurrent weight is 1, so gradients neither vanish nor explode. Each unit has three inputs $(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}, \boldsymbol{c}_{t-1})$ and two outputs $(\boldsymbol{h}_t, \boldsymbol{c}_t)$, and three **gates** (sigmoid valves with values in $(0, 1)$, each with its own weights acting on $[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]$) control writing to and reading from the memory:

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \boldsymbol{g}_t, \qquad \boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t),$$

where the **forget gate** $\boldsymbol{f}_t$ decides what to keep of the old memory, the **input gate** $\boldsymbol{i}_t$ how much of the candidate $\boldsymbol{g}_t$ (a tanh layer) to write, and the **output gate** $\boldsymbol{o}_t$ what to expose as hidden state; $\odot$ is the element-wise product. Variations add **peephole connections** (the gates also see the cell state), couple forget and input gate (write $1 - \boldsymbol{f}_t$ instead of a separate $\boldsymbol{i}_t$), or drop the cell state altogether: the **gated recurrent unit** (GRU) keeps only the hidden-state pipeline.

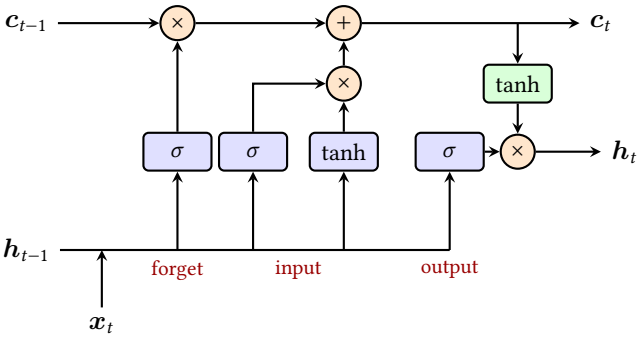


Figure 19: LSTM cell: forget, input and output gates acting on the cell state $\boldsymbol{c}$.

3.3 Transformers

Architecture. The **Transformer** solves most problems of RNNs. It consists of an **encoder** and a **decoder**, each a stack of N identical layers, whose output passes through a linear layer and a softmax (fig. 20); the encoder output is an input of the decoder. Its ingredients are the input and output embeddings, the positional encoding, multi-head attention, feed-forward layers and "Add & Norm" (a residual connection followed by layer normalisation).

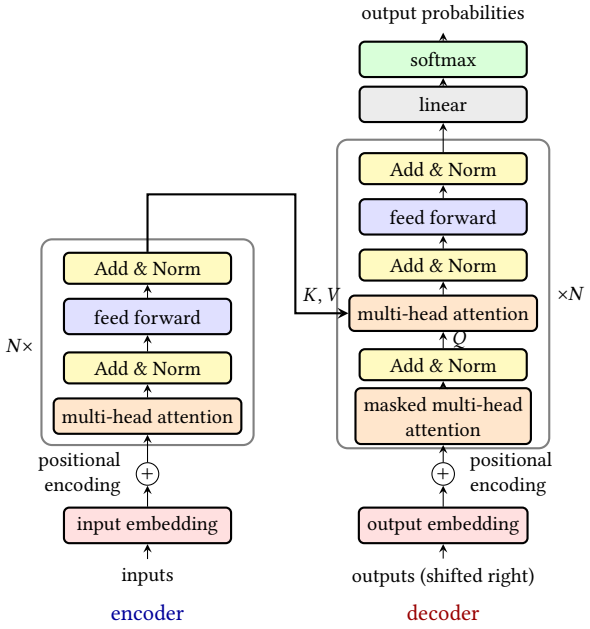


Figure 20: The Transformer: encoder (left) and decoder (right); the encoder output gives keys and values to the decoder's cross-attention.

Embedding and positional encoding. A sentence is split into tokens (words), each token is mapped to its fixed ID in a vocabulary, and the ID to an **embedding** vector of size d_{model}, whose values are learned. Attention by itself ignores word order, so each token also carries its position (words close in the sentence should be treated as close, distant ones as distant): a **positional encoding** of the same size is added to the embedding,

$$\text{PE}(\text{pos}, 2i) = \sin \frac{\text{pos}}{10000^{2i/d_{\text{model}}}}, \qquad \text{PE}(\text{pos}, 2i+1) = \cos \frac{\text{pos}}{10000^{2i/d_{\text{model}}}},$$

for position pos and dimension index i. It is not learned: it is computed once and reused for every sentence, in training and inference. Sine and cosine give a continuous, regular pattern from which relative positions are easy to recognise.

Definition 3.2 (Self-attention)**.** Stack the n token vectors of a sequence as rows of matrices Q (queries), K (keys) and V (values) of size $n \times d_k$. Then

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V,$$

where the softmax acts on each row. The $n \times n$ matrix of attention weights has rows summing to 1, and row j of the $n \times d_k$ result is a weighted mean of the values (fig. 21), which encodes the meaning of token j (embedding), its position (positional encoding) and its interaction with the other tokens.

The names come from a database lookup: a query is compared with all keys, and the values of the best-matching keys are returned, here softly weighted by the scaled dot products. In the simplest form $Q = K = V$ is the input matrix itself, so self-attention relates every word to every other word and

- is **permutation equivariant**: permuting the input tokens permutes the output rows in the same way, leaving each row unchanged;
- has no parameters: the interactions are driven by the embedding and positional encoding (this changes in multi-head attention);
- gives the largest weights typically on the diagonal (each word's dot product with itself). Entries can be manipulated before the softmax: an entry set to $-\infty$ gets weight $e^{-\infty} = 0$.

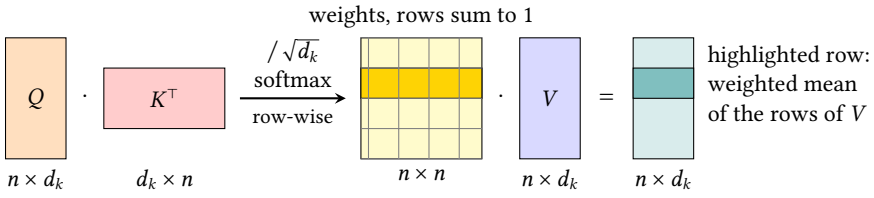


Figure 21: Self-attention: the softmax of the scaled scores weights the rows of V.

Multi-head attention. With h heads and $d_k = d_v = d_{\text{model}}/h$, each head attends in its own learned projection, and the heads are concatenated and projected back:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\, W^O, \qquad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V).$$

The learned matrices W_i^Q, W_i^K, W_i^V, W^O let different heads capture different relations between words (fig. 22).

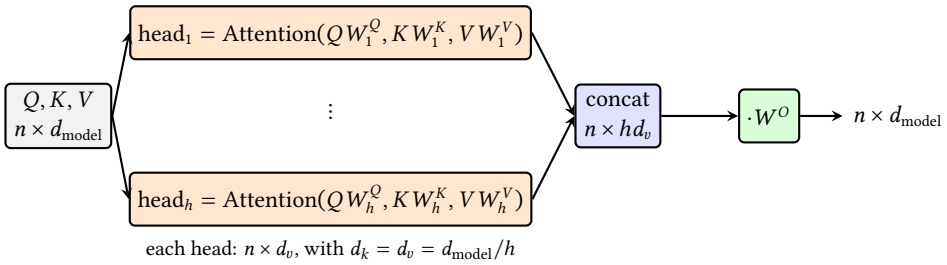


Figure 22: Multi-head attention: h heads in parallel, concatenated and projected by W^O.

Layer normalisation. "Add & Norm" adds the sublayer's input (residual connection) and normalises each item j separately with its own mean μ_j and variance σ_j^2 over its features,

$$\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}},$$

then rescales with learnable parameters γ (multiplicative) and β (additive), since values confined to a fixed range may be too restrictive. Unlike batch normalisation, the statistics are computed per item, not per feature across the batch.

Decoder. The lower part (embedding, positional encoding) is as in the encoder. Its first attention is **masked multi-head attention**: the model is causal, the output at a position may depend only on earlier positions and must not see future words, so all entries above the diagonal of the attention matrix are set to $-\infty$ before the softmax (their weights become 0, fig. 23). The second attention is **cross-attention**: keys and values come from the encoder output, the queries from the masked attention. A final linear layer maps d_{model} to the vocabulary size (the outputs are the **logits**), and a softmax gives the probability of each vocabulary token.

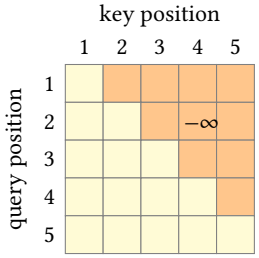


Figure 23: Causal mask: query position j may attend only to key positions $1, \dots, j$.

Training and inference. In training everything happens in one time step: the decoder input is the target sentence preceded by a start token ⟨SOS⟩ ("shifted right"), and the softmax output at every position is compared by a cross-entropy loss with the target followed by ⟨EOS⟩; the mask prevents each position from seeing its own answer. At inference the encoder runs once; the decoder starts from ⟨SOS⟩ and at each step takes the most probable token at the last position, appends it to its input and repeats until it emits ⟨EOS⟩.

Example 3.3 (Translation)**.** Input "⟨SOS⟩ I love you very much ⟨EOS⟩". Training: the decoder reads "⟨SOS⟩ Ti amo molto" and must output "Ti amo molto ⟨EOS⟩" at once. Inference: from "⟨SOS⟩" it outputs "Ti"; from "⟨SOS⟩ Ti" it outputs "amo"; then "molto"; then ⟨EOS⟩, and it stops. The encoder output computed in the first step is reused in all later steps.

Language models. Transformers learn sequence-to-sequence maps (they were first used for machine translation) and are the backbone of large language models (LLMs) such as BERT, ChatGPT, Claude and Llama. Medical NLP tasks fall into two families. **Discriminative tasks** turn text (questions, clinical notes, papers, health records) into structured information, e.g. question answering, or extracting symptoms, drugs and diagnoses and linking them to a terminology such as ICD codes. **Generative tasks** produce new text, e.g. summarising the findings of a radiology report into its impression, or simplifying medical text for readability.

Vision Transformer (ViT). Earlier, attention in vision was used together with CNNs or to replace some of their components. ViT removes convolutions entirely and applies a pure Transformer encoder to image patches treated like tokens (fig. 24):

1. split the image into patches (e.g. 16×16 pixels) and flatten each one;
2. project each patch linearly to dimension D;
3. prepend a learnable `[class]` embedding and add learned position embeddings;
4. pass the sequence through L Transformer encoder layers;
5. an MLP head reads the output at the `[class]` position and predicts the class.

It is trained on image classification in a supervised way, and reached excellent results compared with the best CNNs at lower training cost.

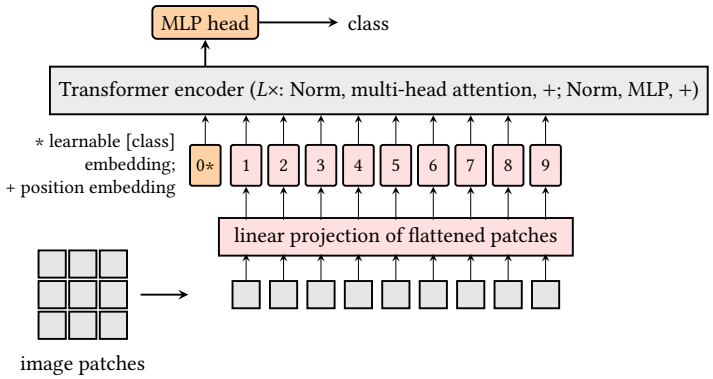


Figure 24: Vision Transformer: patches become tokens; the MLP head classifies from the `[class]` token.

Pre-training and fine-tuning. ViT is typically pre-trained on large datasets (also self-supervised) and fine-tuned on smaller downstream tasks: the pre-trained prediction head is replaced by a zero-initialised $D \times K$ layer for K downstream classes. Fine-tuning at a higher resolution than pre-training often helps; the patch size stays the same, so the sequence gets longer. The Transformer handles any sequence length, but the pre-trained position embeddings then lose their meaning, which is fixed by 2D interpolation according to their location in the original image. This resolution adjustment and the patch extraction are the only places where an inductive bias about the 2D structure of images is put in by hand.

Inspecting a trained ViT shows that it attends to the image regions semantically relevant for the class, and that the learned position embeddings of patches in the same row or column are similar, i.e. the model learns the 2D layout.

Swin Transformer. ViT keeps the same coarse resolution (patches of 16× the pixel size) in every layer. The **Swin Transformer** uses a hierarchical, CNN-like pyramid that merges patches and so downsamples the feature maps gradually (4×, 8×, 16×, …), which helps dense downstream tasks such as object detection and semantic segmentation (fig. 25). Self-attention is computed only within local windows (W-MSA), and successive blocks shift the windows (SW-MSA) so that information flows between neighbouring windows.

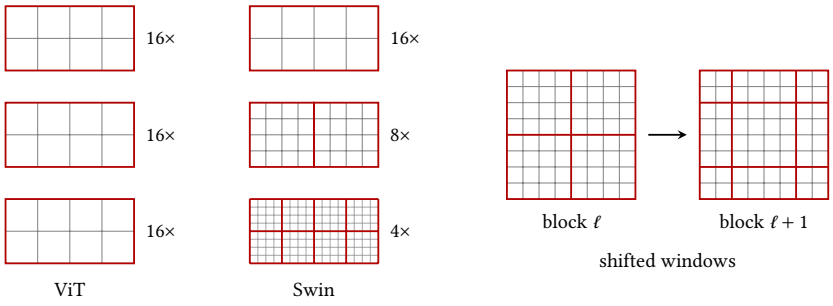


Figure 25: ViT keeps 16× patches throughout; Swin merges patches (4× to 16×) and attends within local (red) windows, shifted in the next block.

Case study: COVID-19 on chest X-rays. A multi-task ViT for diagnosis and severity quantification, made of

- a **shared part**: a pre-trained CNN backbone (a DenseNet) extracts low-level chest X-ray features, which become the tokens of a Transformer encoder with a `[class]` token;
- two **task-specific heads**: a classification head on the `[class]` output (normal, other infection, COVID-19), and a map head whose severity map, restricted to the lungs by a mask, is max-pooled over lung regions into severity scores trained against the radiologists' region-wise labels.